

lab04

Generative AI in Practice

You'll learn: how to use pre-trained generative models for text summarization, creative writing, code generation, image synthesis, and data augmentation — plus basic evaluation of outputs.

Prereqs: Python, Google Colab (GPU optional for image tasks).

Deliverables: screenshots of outputs + short reflections for each lab.

Lab 1 — Text Summarization with a Pretrained LLM (20–25 mins)

Goal: Use a Hugging Face model to condense long text into short, clear summaries.

Setup

In []:

```
#!pip install -q torch --index-url
https://download.pytorch.org/whl/cpu
#!pip install transformers
from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-
cnn")
```

Steps

1. Paste a long article/essay paragraph.
2. Run:

In []:

```
text = """
Why Self-Attention
In this section we compare various aspects of self-attention layers
to the recurrent and convolu-
tional layers commonly used for mapping one variable-length sequence
of symbol representations
(x1, ..., xn) to another sequence of equal length (z1, ..., zn),
with xi, zi ∈ Rd, such as a hidden
```

layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [11]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [31] and byte-pair [25] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(\log k(n))$ in the case of dilated convolutions [15], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k . Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable

convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model. As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

```
summary = summarizer(text, max_length=120, min_length=40,
do_sample=False)
print(summary[0]["summary_text"])
```

3. Compare outputs with different max_length/min_length.
4. Try domain-specific text (tech, news, academic).

Checkpoint: screenshot of at least 2 summaries.

Reflection: Did shorter summaries miss context? Did longer ones add redundancy?

Lab 2 — Creative Generation: Stories & Marketing Copy (20–25 mins)

Goal: Practice shaping outputs for creativity and business use.

Setup

In []:

```
gen = pipeline("text-generation", model="gpt2", device=-1)
```

Steps

1. Prompt: "Write a short bedtime story about a robot learning kindness."
2. Prompt: "Create 3 catchy product taglines for an eco-friendly laptop sleeve brand."
3. Adjust parameters: temperature (0.5 vs 1.0), max_new_tokens (50 vs 100).
4. Compare how creativity changes.

Checkpoint: 1 story + 1 set of taglines.

Reflection: Which parameter setting gave the best balance of coherence vs creativity?

In []:

```
story = gen(
```

```

    "Write a short bedtime story about a robot learning kindness.",
    temperature=0.9, # more creative
    top_p=0.9, # nucleus sampling for variety
    repetition_penalty=1.2, # discourages loops
    max_new_tokens=120,
)
print(story[0]["generated_text"])

```

In []:

```

story = gen(
    "Create 3 catchy product taglines for an eco-friendly laptop
    sleeve brand.",
    temperature=0.9, # more creative
    top_p=0.9, # nucleus sampling for variety
    repetition_penalty=1.2, # discourages loops
    max_new_tokens=120,
)
print(story[0]["generated_text"])

```

Lab 3 — Code Generation & Debugging (25–30 mins)¶

Goal: Use generative AI to produce useful code snippets and validate them.

Setup

In []:

```

coder = pipeline("text-generation", model="gpt2", device=-1)

```

Steps

1. Prompt: "Write Python code to calculate cosine similarity between two vectors."
2. Copy result into Colab cell → run it.
3. Debug errors (if any).
4. Extend: "Now make it read vectors from a CSV file."
5. Run final code on a toy CSV.

Checkpoint: screenshot of working code output.

Reflection: What mistakes did the model make? How easy were they to fix?

In []:

```

code = coder(
    "Write Python code to calculate cosine similarity between two
    vectors.",
    temperature=0.7,
    max_new_tokens=120,
)

```

```
print(code[0]["generated_text"])
```

Lab 4 — Image Generation with Diffusion (30–40 mins)¶

Goal: Generate and visualize synthetic images with Stable Diffusion.

Setup (GPU recommended in Colab)

In []:

```
#!pip install -q diffusers transformers accelerate safetensors
import torch
from diffusers import StableDiffusionPipeline

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", dtype=dtype
).to(device)
```

Steps

1. Prompt: "A futuristic city skyline at sunset, digital art style."
2. Generate and display image.

In []:

```
img = pipe(
    "A futuristic city skyline at sunset, digital art style",
    guidance_scale=7.5
).images[0]
img.show()
```

3. Try 3 more prompts (product, nature, abstract).
4. Vary guidance_scale (5.0 vs 8.5) to see differences.

Checkpoint: 2–3 generated images.

Reflection: Which prompts produced the most realistic results? Which were less reliable?

Lab 5 — Data Augmentation for ML (20–25 mins)¶

Goal: Use text generation to expand a dataset (e.g., sentiment analysis).

Steps

1. Define 5 "positive" reviews and 5 "negative" reviews.
2. Prompt: "Generate 5 more unique positive product reviews (2-3 sentences each)."
3. Save augmented dataset to CSV.

In []:

```
import pandas as pd
```

```
reviews = {
    "text": [
        "Great laptop sleeve, very durable!",
        "Love the eco-friendly material and sleek design.",
        "Super soft fabric and fits perfectly.",
        "The zipper is smooth and feels sturdy.",
        "Excellent value for the price.",
        "Not worth the money, tore quickly.",
        "The stitching came undone after a week.",
        "Looks cheap and feels flimsy.",
        "Too small for my laptop, disappointed.",
        "Smells weird and feels low quality.",
    ],
    "label": [
        "positive",
        "positive",
        "positive",
        "positive",
        "positive",
        "negative",
        "negative",
        "negative",
        "negative",
        "negative",
    ],
}
```

```
df = pd.DataFrame(reviews)
print(df.head())
```

In []:

```
from transformers import pipeline
```

```
gen = pipeline("text-generation", model="gpt2-medium", device=-1)
```

```
prompt = (
    "Generate 5 unique positive product reviews about a laptop sleeve. "
    "Each review should be 2-3 sentences long and sound natural."
)
```

```
generated = gen(prompt, temperature=0.9, max_new_tokens=150)[0]
["generated_text"]
print(generated)
```

In []:

```
# Example manual split - adjust as needed
new_reviews = [
    "This sleeve is stylish and protective, I use it every day!",
    "Fantastic quality and eco packaging, very satisfied.",
    "Holds my laptop snugly and the color looks amazing.",
    "Soft material and strong seams – feels premium.",
    "Lightweight yet durable, totally worth the price!",
]

new_df = pd.DataFrame({"text": new_reviews, "label": ["positive"] *
len(new_reviews)})
```

```
# Combine with original dataset
augmented_df = pd.concat([df, new_df], ignore_index=True)
```

In []:

```
augmented_df.to_csv("augmented_reviews.csv", index=False)
print("Saved augmented dataset to augmented_reviews.csv")
```

In []:

```
loaded = pd.read_csv("augmented_reviews.csv")
print(loaded["label"].value_counts())
print(loaded.sample(5))
```

4. Load and check distribution.

Checkpoint: screenshot of CSV with ≥ 10 rows. **Reflection:** Did generated data look realistic and balanced?

Grading Rubric (10 pts)

- Summarization (2 pts)
- Creative generation (2 pts)
- Code generation/debugging (2 pts)
- Image generation (2 pts)
- Data augmentation reflection (2 pts)